

The list data type is an ordered sequence of elements that can be comprised of strings, numbers, or even other lists. Lists are mutable and use zero-based indexing, meaning that the first element of the list is at index zero.

Here is the basic syntax for a list:

```
cities = ['Los Angeles', 'London', 'Tokyo']
```

To access an element from the `cities` list, you can reference its index number in the sequence. Here is an example of accessing the first element of the `cities` list:

```
cities = ['Los Angeles', 'London', 'Tokyo']  
cities[0] # 'Los Angeles'
```

Negative indexing is used to access elements starting from the end of the list instead of the beginning at index `0`. To access the last element of any list, you can use `-1` like this:

```
cities = ['Los Angeles', 'London', 'Tokyo']  
cities[-1] # 'Tokyo'
```

Another way to create a list is to use the `list()` constructor. The `list()` constructor is used to convert an iterable into a list like this:

```
developer = 'Jessica'  
list(developer) # ['J', 'e', 's', 's', 'i', 'c', 'a']
```

An iterable is a special type of object that can be looped over one item at a time. You will learn more about loops in Python later on.

To get the total number of elements in a list, you can use the `len()` function like this:

```
numbers = [1, 2, 3, 4, 5]  
len(numbers) # 5
```

If you wanted to update a value at a particular index, you can do something like this:

```
programming_languages = ['Python', 'Java', 'C++', 'Rust']
programming_languages[0] = 'JavaScript'
print(programming_languages) # ['JavaScript', 'Java', 'C++', 'Rust']
```

Since lists are mutable, you can update any element in the list as long as you pass in a valid index number. If you pass in an index (either positive or negative) that is out of bounds for the list, then you will receive an `IndexError`:

```
programming_languages = ['Python', 'Java', 'C++', 'Rust']
programming_languages[10] = 'JavaScript'
```

```
"""
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 2, in <module>
```

```
IndexError: list assignment index out of range
```

```
"""
```

If you want to remove an element from a list you can use the `del` keyword like this:

```
developer = ['Jane Doe', 23, 'Python Developer']
del developer[1]
print(developer) # ['Jane Doe', 'Python Developer']
```

Sometimes it is helpful to check if an element is inside the list. To do that, you can use the `in` keyword like this:

```
programming_languages = ['Python', 'Java', 'C++', 'Rust']

'Rust' in programming_languages # True
'JavaScript' in programming_languages # False
```

Sometimes it is common to have lists nested inside of other lists like this:

```
developer = ['Alice', 25, ['Python', 'Rust', 'C++']]
```

In this example, we have one nested list containing three popular programming languages. To access the nested list, you will need to access it using index `2` since lists are zero based indexed:

```
developer = ['Alice', 25, ['Python', 'Rust', 'C++']]
developer[2] # ['Python', 'Rust', 'C++']
```

Then to access the second language from that nested list, you will need to access it using index `1` like this:

```
developer = ['Alice', 25, ['Python', 'Rust', 'C++']]
developer[2][1] # 'Rust'
```

Another common technique used with lists is unpacking values.

Unpacking values from a list is a technique used to assign values from a list to new variables. Here is an example of unpacking a `developer` list into new variables called `name`, `age` and `job`.

```
developer = ['Alice', 34, 'Rust Developer']
name, age, job = developer

print(name) # 'Alice'
print(age) # 34
print(job) # 'Rust Developer'
```

Here, `name` has the value `'Alice'`, `age` has the value `34`, and `job` has the value `'Rust Developer'`.

If you need to collect any remaining elements from a list, you can use the asterisk (`*`) operator like this:

```
developer = ['Alice', 34, 'Rust Developer']
name, *rest = developer
```

```
print(name) # 'Alice'
print(rest) # [34, 'Rust Developer']
```

In this example, `name` will still have the value `'Alice'`, and `rest` is a list of two items: the number `34` and the string `'Rust Developer'`.

If the numbers of variables on the left side of the assignment operator doesn't match the total numbers of items in the list, then you will receive a `ValueError`:

```
developer = ['Alice', 34, 'Rust Developer']
name, age, job, city = developer

"""
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
ValueError: not enough values to unpack (expected 4, got 3)
"""
```

The last concept we will look at is the slice operator (`:`). Similar to strings, you can access portions of a list by using the slice operator like this:

```
desserts = ['Cake', 'Cookies', 'Ice Cream', 'Pie', 'Brownies']
desserts[1:4] # ['Cookies', 'Ice Cream', 'Pie']
```

In this example, the start index is `1` since that points to the second item in the list. Then we use the slice operator followed by an end index of `4`, which includes everything up to (but not including), the item at that index.

Another thing you can do with the slice operator `:` is specify a step interval which determines how much to increment between the indices. Let's say you had a list of numbers like this:

```
numbers = [1, 2, 3, 4, 5, 6]
```

If you wanted to extract a list of just even numbers, you can use the slicing operator like this:

```
numbers = [1, 2, 3, 4, 5, 6]
numbers[1::2] # [2, 4, 6]
```

The first even number is at index `1`, so that will be the start index. Since we want to go through the end of the list, then we omit the end index. Lastly, we specify `2` for the optional step interval so it will only increment by `2` instead of the default `1`.

Lists are a useful and flexible data structure that you will use a lot in your Python programs. In the next lesson, you will learn about common methods that you can use with lists.

What Are Some Common Methods Used for Lists?

In the previous lesson, you were introduced to the list data type and learned how to access elements from a list as well as list slicing. In this lesson, you will continue to learn about lists and some common methods associated with them like `append()`, `pop()`, and `sort()`.

The first method we will look at is the `append()` method. This is used to add an item to the end of the list. Here is an example of using the `append()` method to add the number `6` to list of numbers:

```
numbers = [1, 2, 3, 4, 5]
numbers.append(6)
print(numbers) # [1, 2, 3, 4, 5, 6]
```

If you want to add one list at the end of another, you can also use the `append()` method like this:

```
numbers = [1, 2, 3, 4, 5]
even_numbers = [6, 8, 10]

numbers.append(even_numbers)
```

```
print(numbers) # [1, 2, 3, 4, 5, [6, 8, 10]]
```

Notice how the entire `even_numbers` list is nested inside of the `numbers` list.

But if you want to add all of the individual numbers from the `even_numbers` list at the end of the `numbers` list, then you can use the `extend()` method.

The `extend()` method is similar to the `append()` method, but with `extend()` you can add multiple elements from one list to another. Here's an example of adding the numbers `6`, `8`, and `10` from one list to the end of the `numbers` list:

```
numbers = [1, 2, 3, 4, 5]
even_numbers = [6, 8, 10]

numbers.extend(even_numbers)
print(numbers) # [1, 2, 3, 4, 5, 6, 8, 10]
```

As you can see, the nested list is gone and it's just a list of numbers.

To insert an element at a specific index in a list, you can use the `insert()` method. This method accepts two arguments: the index where you wish to insert the new item and the item you want to insert.

Here is an example of using the `insert()` method:

```
numbers = [1, 2, 3, 4, 5]
numbers.insert(2, 2.5)

print(numbers) # [1, 2, 2.5, 3, 4, 5]
```

The following code will insert the number `2.5` at index `2` in the `numbers` list.

If you want to remove an element from a list, you can use the `remove()` method. The `remove()` method takes the value of the element to remove as an argument:

```
numbers = [10, 20, 30, 40, 50, 50]
numbers.remove(50)
```

```
print(numbers) # [10, 20, 30, 40, 50]
```

It is important to note that this method will only remove the first occurrence of an item. Not all of them:

```
numbers = [10, 20, 30, 40, 50, 50, 50]
numbers.remove(50)
```

```
print(numbers) # [10, 20, 30, 40, 50, 50]
```

To remove an element at a specific index in the list, you can use the `pop()` method like this:

```
numbers = [1, 2, 3, 4, 5]
numbers.pop(1) # The number 2 is returned
```

If you don't specify an element for the `pop()` method, then the last element is removed.

```
numbers = [1, 2, 3, 4, 5]
numbers.pop() # The number 5 is returned
```

If you need to empty the list, then you can use the `clear()` method like this:

```
numbers = [1, 2, 3, 4, 5]
numbers.clear()

print(numbers) # []
```

The next method we will take a look at is the `sort()` method. This method is used to sort the elements in place. Here is an example of sorting a random list of numbers in place:

```
numbers = [19, 2, 35, 1, 67, 41]
numbers.sort()
```

```
print(numbers) # [1, 2, 19, 35, 41, 67]
```

In contrast to the `sort()` method, there is the `sorted()` function which works for any iterable and returns a new sorted list instead of modifying the original list. For example:

```
numbers = [19, 2, 35, 1, 67, 41]
sorted_numbers = sorted(numbers)
```

```
print(numbers) # [19, 2, 35, 1, 67, 41]
```

```
print(sorted_numbers) # [1, 2, 19, 35, 41, 67]
```

As a reminder, an iterable is a special type of object that you can loop over, allowing you to access each item one at a time. You'll learn more about how loops work in Python in a future lesson.

Both the `sort()` method and `sorted()` function accept optional `key` and `reverse` parameters. You will learn more about these optional parameters in a future lesson when you learn about tuples. You'll also learn more about other built-in functions like `sorted()` in future lessons.

The next method we will take a look at is the `reverse()` method. This method, will reverse a list of elements in place like this:

```
numbers = [6, 5, 4, 3, 2, 1]
numbers.reverse()
```

```
print(numbers) # [1, 2, 3, 4, 5, 6]
```

The last method we will take a look at is the `index` method. This is used to find the first index where an element can be found in a list. Here is an example of using the `index` method to find the language `'Java'` in a `programming_languages` list:

```
programming_languages = ['Rust', 'Java', 'Python', 'C++']
programming_languages.index('Java') # 1
```


If the element cannot be found, then Python throws a `ValueError`:

```
programming_languages = ['Rust', 'Java', 'Python', 'C++']
programming_languages.index('JavaScript')
```

```
"""
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in <module>
```

```
ValueError: 'JavaScript' is not in list
```

```
"""
```

There are a few more methods for Python lists, but this initial list of methods is a good place to start.

What Are Tuples and How Do They Work?

A tuple is a Python data type used to create an ordered sequence of values. Tuples can contain a mixed set of data types like this:

```
developer = ('Alice', 34, 'Rust Developer')
```

Tuples are similar to lists, but while lists are a mutable data type, tuples are immutable. This means that the elements in a tuple cannot be changed once it's created.

If you try to update one of the items in the tuple, you will get a `TypeError`:

```
programming_languages = ('Python', 'Java', 'C++', 'Rust')
programming_languages[0] = 'JavaScript'
```

```
"""
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 2, in <module>
```

```
TypeError: 'tuple' object does not support item assignment
"""
```

To access an element from a tuple, you can use bracket notation and the index number:

```
developer = ('Alice', 34, 'Rust Developer')
developer[1] # 34
```

If you need to access elements starting from the end of a tuple, then you can use negative indexing. Here is an example of using a negative index to access the second to last element in a tuple:

```
numbers = (1, 2, 3, 4, 5)
numbers[-2] # 4
```

If you try to pass in an index number that exceeds or equals the length of the tuple, then you will receive an `IndexError` like this:

```
numbers = (1, 2, 3, 4, 5)
numbers[7]

"""
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
IndexError: list index out of range
"""
```

Another way to create a tuple is by using the `tuple()` constructor like this:

```
developer = 'Jessica'
tuple(developer) # ('J', 'e', 's', 's', 'i', 'c', 'a')
```

For the `tuple()` constructor, you can pass in different iterables like strings, lists and even other tuples.

To check if an item is in a tuple, you can use the `in` keyword like this:

```
programming_languages = ('Python', 'Java', 'C++', 'Rust')

'Rust' in programming_languages # True
'JavaScript' in programming_languages # False
```

You can also unpack items from a tuple just like you did with lists:

```
developer = ('Alice', 34, 'Rust Developer')
name, age, job = developer

print(name) # 'Alice'
print(age) # 34
print(job) # 'Rust Developer'
```

In this example, `name` has the value `'Alice'`, `age` has the value `34`, and `job` has the value `'Rust Developer'`.

If you need to collect any remaining elements from a tuple, you can use the asterisk (`*`) operator like this:

```
developer = ('Alice', 34, 'Rust Developer')
name, *rest = developer

print(name) # 'Alice'
print(rest) # [34, 'Rust Developer']
```

Here, `name` has the value `'Alice'`, and `rest` is a list comprised of the number `34` and the string `'Rust Developer'`.

Just like with a list, you can use the slice operator on a tuple to extract a portion of it. Here is an example of extracting the items `'pie'` and `'cookies'` into a separate tuple:

```
desserts = ('cake', 'pie', 'cookies', 'ice cream')
```

```
desserts[1:3] # ('pie', 'cookies')
```

Remember that the first number represents the starting index for the extraction while the second number represents the ending index. But note that the item at the ending index is not included in the extracted tuple.

If you need to remove an item from a tuple, that isn't possible because tuples are immutable. So this example, will produce an error:

```
developer = ('Jane Doe', 23, 'Python Developer')
del developer[1]

"""
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
TypeError: "tuple" object doesn't support item deletion
"""
```

So when might you use a tuple over a list?

If you need a dynamic collection of elements where you can add, remove and update elements, then you should use a list. If you know that you are working with a fixed and immutable collection of data, then you should use a tuple.

What Are Some Common Methods for Tuples?

In the previous lesson, you learned how to work with the tuple data type. In this lesson, you'll learn about some common methods you'll use when working with tuples.

The first method we will cover is `count()`. This method is used to determine how many times an item appears in a tuple. Here is an example of checking how many times the string `Rust` appears in a tuple named `programming_languages`:

```
programming_languages = ('Rust', 'Java', 'Python', 'C++',  
                          'Rust')
```

```
programming_languages.count('Rust') # 2
```

Since `Rust` appears twice in the tuple, the `count()` method returns the number `2`. If the specified item in the `count()` function is not present at all in the tuple, then the return value is `0`:

```
programming_languages = ('Rust', 'Java', 'Python', 'C++',  
                          'Rust')
```

```
programming_languages.count('JavaScript') # 0
```

If no arguments are passed into the `count()` function, then Python raises a `TypeError`:

```
programming_languages = ('Rust', 'Java', 'Python', 'C++',  
                          'Rust')
```

```
programming_languages.count()
```

```
"""
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 2, in <module>
```

```
TypeError: tuple.count() takes exactly one argument (0 given)
```

```
"""
```

The next method we will look at is the `index()` method. This method is used to find the index where a particular item is present in a tuple. Here is an example of using the `index()` method to find the index for the string `Java`:

```
programming_languages = ('Rust', 'Java', 'Python', 'C++',  
                          'Rust')
```

```
programming_languages.index('Java') # 1
```

If the specified item cannot be found, then Python raises a `ValueError`:

```
programming_languages = ('Rust', 'Java', 'Python', 'C++',  
                          'Rust')
```

```
programming_languages.index('JavaScript')
```

```
"""
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 2, in <module>
```

```
ValueError: tuple.index(x): x not in tuple
```

```
"""
```

Another thing you can do with the `index()` method is to pass in optional start and stop index arguments. Here is an example of passing in an optional start index:

```
programming_languages = ('Rust', 'Java', 'Python', 'C++',  
                          'Rust', 'Python')
```

```
programming_languages.index('Python', 3) # 5
```

In this example, we are specifying where to start searching for the string `Python`. By passing in the number `3` as the second argument to the `index()` function, we are specifying to start searching at index `3`. Since `Python` appears twice in the tuple, the `index()` function will return index `5` instead of index `2` because of the use of the optional start index argument.

You can also pass in an optional stop index. Here is a modified example of specifying a start and stop index:

```
programming_languages = ('Rust', 'Java', 'Python', 'C++',  
                          'Rust', 'Python', 'JavaScript', 'Python')
```

```
programming_languages.index('Python', 2, 5) # 2
```

Now the result is index `2` because we are starting the search at index `2`, and searching up to, but not including, index `5`.

Another commonly used function used with tuples is the `sorted()` function. In a previous lesson you learned about the `sort()` method for lists. Well, the `sorted()` function can be used on any iterable including tuples.

Here is an example of creating a new list of numbers using the `sorted()` function:

```
numbers = (13, 2, 78, 3, 45, 67, 18, 7)
sorted(numbers) # [2, 3, 7, 13, 18, 45, 67, 78]
```

The `sorted()` function will always create a new list of the sorted values. This differs from the `sort()` method which sorts the elements of a list in place and does not return a new list.

If you need to customize the sorting behavior for an iterable, you can use the optional `reverse` and `key` arguments. Here is an example of using `key` argument to sort items in a tuple by length:

```
programming_languages = ('Rust', 'Java', 'Python', 'C++',
                          'Rust', 'Python')
sorted(programming_languages, key=len)
```

```
# Result
```

```
# ['C++', 'Rust', 'Java', 'Rust', 'Python', 'Python']
```

If you want to create a new list of values in reverse order, then you can use the `reverse` argument like this:

```
programming_languages = ('Rust', 'Java', 'Python', 'C++',
                          'Rust', 'Python')
```

```
print(sorted(programming_languages, reverse=True))
```

```
# Result
```

```
# ['Rust', 'Rust', 'Python', 'Python', 'Java', 'C++']
```

Tuples are a common data type in Python. Understanding how to work with them, along with some helpful methods and functions, will help you write more efficient code.